

A. Adventure Time

Author: Andrei Costin-Costantinescu
Solved by: 0/21
First to solve: N/A

Small k ($k \leq 6$)

When k is small, the natural state is the *window* of the last $k-1$ decisions.

Define a DP over prefixes and pair counts:

$\text{dp}[i][t][\text{mask}]$ = number of ways to complete the first i seats with exactly t buddy pairs,

where $\text{mask} \in [0, 2^{k-1})$ encodes which of seats $i - (k-1), \dots, i-1$ are occupied (bit 0 = seat $i-1$).
Transitions for seat i :

- If $s_i \in \{0, ?\}$, we may place 0: shift the window, no new pairs

$$\text{dp}[i+1][t][(\text{mask} \ll 1) \& (2^{k-1}-1)] += \text{dp}[i][t][\text{mask}].$$

- If $s_i \in \{1, ?\}$, we may place 1: new pairs equal the number of 1-bits currently in the window,

$$\text{dp}[i+1][t+\text{popcount}(\text{mask})][((\text{mask} \ll 1)+1) \& (2^{k-1}-1)] += \text{dp}[i][t][\text{mask}].$$

Initialize $\text{dp}[0][0][0] = 1$ and cap t at x . The answer is $\sum_{\text{mask}} \text{dp}[n][x][\text{mask}]$. This runs in $O(n \cdot x \cdot 2^{k-1})$ time and $O(x \cdot 2^{k-1})$ memory.

Large k ($k \geq \frac{n}{2}$)

Partition seats into blocks of length k : $\mathcal{B}_1 = [1..k]$ and $\mathcal{B}_2 = [k+1..\min(2k, n)]$ (the second block may be shorter). Process seats by residues $r = 1, 2, \dots, k$; for each residue, handle block 1 then block 2 if its seat exists:

$$1, k+1, 2, k+2, \dots, k, 2k \quad (\text{skip indices } > n).$$

Let $\mathbf{c} = (c_1, c_2)$, where c_b is the number of 1's already placed in block b among residues processed so far. Define

$\text{dp}[\mathbf{c}][t]$ = number of ways after all seats processed so far, with counters $\mathbf{c}$ and t buddy pairs.

Initialize $\text{dp}[(0,0)][0] = 1$. For each residue $r = 1, \dots, k$ do:

(A) Seat $i = r$ in $\mathcal{B}_1$ (if $i \leq n$). Use a table dp^* and update

$$\begin{aligned} \text{if } s_i \in \{0, ?\} : & \quad \text{dp}^*[(c_1, c_2)][t] += \text{dp}[(c_1, c_2)][t], \\ \text{if } s_i \in \{1, ?\} : & \quad \text{dp}^*[(c_1+1, c_2)][t + (c_1+c_2)] += \text{dp}[(c_1, c_2)][t]. \end{aligned}$$

(B) Seat $j = k+r$ in $\mathcal{B}_2$ (only if $j \leq n$). Fold dp^* back into dp via

$$\begin{aligned} \text{if } s_j \in \{0, ?\} : & \quad \text{dp}[(c_1, c_2)][t] += \text{dp}^*[(c_1, c_2)][t], \\ \text{if } s_j \in \{1, ?\} : & \quad \text{dp}[(c_1, c_2+1)][t + c_2] += \text{dp}^*[(c_1, c_2)][t]. \end{aligned}$$

If $j > n$ (no seat in $\mathcal{B}_2$ at residue r), simply set $\text{dp} \leftarrow \text{dp}^*$.

At $i = r \in \mathcal{B}_1$, previously processed buddies within distance $< k$ are exactly the seats of $\mathcal{B}_1$ and $\mathcal{B}_2$ with residues $< r$, contributing $c_1 + c_2$. At $j = k+r \in \mathcal{B}_2$, previously processed seats in $\mathcal{B}_1$ have distance $\geq k$, so only earlier residues in $\mathcal{B}_2$ contribute, i.e. c_2 . The result is

$$\sum_{0 \leq c_1 \leq \min(k, n)} \sum_{0 \leq c_2 \leq \max(0, n-k)} \text{dp}[(c_1, c_2)][x]. \quad \text{The complexity is } O(n \cdot x \cdot k^2).$$

General solution

For arbitrary k we can generalize the interleaving trick to *blocks of length k* . Partition the seats into $B = \lceil \frac{n}{k} \rceil$ contiguous blocks of (last block possibly shorter):

$$\mathcal{B}_1 = [1..k], \mathcal{B}_2 = [k+1..2k], \dots, \mathcal{B}_B.$$

Process seats in increasing remainder modulo k , i.e., by *columns* of this block layout:

$$1, 1+k, 1+2k, \dots; \quad 2, 2+k, 2+2k, \dots; \quad \dots \quad k, 2k, 3k, \dots$$

Fix the current seat $i = (b-1)k + r$ (block b , residue r). Among the *already processed* seats, those at distance $< k$ from i are *exactly*:

- seats of the *same block* $\mathcal{B}_b$ with residues $r' < r$; and
- seats of the *next block* $\mathcal{B}_{b+1}$ with residues $r' < r$ (if $b+1 \leq B$).

Indeed, for $j \in \mathcal{B}_{b+1}$ with residue r' we have $j - i = k + (r' - r) < k \iff r' < r$, while for $j \in \mathcal{B}_{b-1}$ with residue $r' \leq r$ we get $i - j = k + (r - r') \geq k$, so these are not buddies.

Let c_b and c_{b+1} denote the numbers of 1's already placed in blocks b and $b+1$ *among residues $< r$* . With the above processing order, these are precisely the cumulative block counts stored by the DP. Thus, placing a 1 at i increases the pair count by

$$\Delta = c_b + c_{b+1} \quad (\text{with } c_{b+1} = 0 \text{ if } b+1 > B).$$

DP state and transition. The state remains $\text{dp}[\mathbf{c}][t]$ with $\mathbf{c} = (c_1, \dots, c_B)$. At seat $i \in \mathcal{B}_b$:

- If $s_i \in \{0, ?\}$:

$$\text{dp}'[\mathbf{c}][t] += \text{dp}[\mathbf{c}][t].$$

- If $s_i \in \{1, ?\}$:

$$\text{dp}'[c_1, \dots, c_b+1, \dots, c_B][t + c_b + c_{b+1}] += \text{dp}[\mathbf{c}][t].$$

The number of states is $\prod_{b=1}^B (|\mathcal{B}_b| + 1) \leq (k+1)^B = \tilde{O}(k^{\frac{n}{k}})$, yielding $O(k^{\frac{n}{k}} \cdot n \cdot x)$ time and memory.

Choosing the faster DP. Compare the small- k mask DP $O(2^{k-1})$ against the block DP $O(k^{\frac{n}{k}})$. Run the mask DP for $k \leq P$ and the block DP otherwise, where P balances $2^P \approx P^{n/P}$. Equivalently, P solves $P^2 \approx n \log_2 P$, giving

$$\sqrt{n} \leq P \leq \sqrt{n \log_2 n}.$$

Thus the overall complexity is $O(2^P \cdot n \cdot x)$ with $P = \Theta(\sqrt{n \log n})$ in the worst case, which is easily fast for $n \leq 50$.

B. Ben 10

Author: Alexandru Gheorghies
Solved by: 21/21
First to solve: *Kindergarten Timelimit*

Since Ben 10's vines cannot expand downwards, we can force the vine to first occupy the first row, then the second row, and so on.

Let $f(n, m)$ be the number of possible vines for some values of n and m . Due to the aforementioned property, $f(n, m) = f(2, m)^{n-1}$.

$f(2, m)$ is equivalent to the number of ways of partitioning an array of length m into several disjoint subarrays, and choosing a "leader" for each subarray.

We will consider:

- $dp[i][0]$ - the number of ways of partitioning an array of length i into several disjoint subarrays, and choosing a "leader" for each subarray.
- $dp[i][1]$ - the number of ways of partitioning an array of length m into several disjoint subarrays, and choosing a "leader" for each subarray except for the last one.

Naturally, the answer will be found in $dp[m][0]$.

We have the following transitions:

1. $dp[1][0] = dp[1][1] = 1$
2. $dp[i][0] = 2 \cdot dp[i-1][0] + dp[i-1][1]$
3. $dp[i][1] = dp[i-1][0] + dp[i-1][1]$

$$\begin{aligned} \text{As such, } f(2, m) &= \left[\begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}^{m-1} \right]_{1,1} = \left[\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \cdot \left[\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^2 \right]^{m-1} \right]_{1,1} = \\ &= \left[\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{2 \cdot m - 1} \right]_{1,1} = \text{fibonacci}(2 \cdot m - 1). \end{aligned}$$

Therefore, the answer for this problem is: $\left(\text{fibonacci}(2 \cdot m - 1) \right)^{n-1}$.

Time complexity per query: $O(\log(m) + \log(n))$

C. Courage the Cowardly Dog

Author: Andrei Boacă
Solved by: 12/21
First to solve: *KoRoVa!*

It is obvious that the *Courage* function is just a binary search that stops if it finds the sought number or if the search interval becomes empty.

First, let's try to solve a single query. Suppose we know x , and we want to find the sum of $Courage(p, 1, n, x)$ across all permutations p .

Consider the index where the binary search process terminates when searching for x , say i . Notice that the path to i is uniquely determined: It has length $O(\log(n))$, and is obtained by an unique sequence of left/right moves, depending on how p_{middle} compares to x . In particular, if we place elements that compare correctly at these positions, we can always force the binary search to terminate at i , regardless of what the rest of p looks like.

Suppose we require L elements strictly smaller than x , G elements strictly larger than x , and E elements equal to x (note that E is always 0 or 1, depending on how the search terminates). Let's count the contribution of each integer from 1 to N to this configuration.

Consider some $1 \leq y < x$. In how many permutations does y occur in the path to i when searching for x ?

A bit of combinatorics tells us that:

- y can be placed in any of the L positions for "less than" elements; this is done in L ways.
- For the remaining $L - 1$ positions, we can choose and place them in $\binom{x-2}{L-1} \cdot (L - 1)!$ ways.
- The "greater than" positions are similarly filled in $\binom{N-x}{G} \cdot G!$ ways.
- If $E = 1$ we must place x at index i , otherwise we don't really care.
- Finally, all the remaining elements can be permuted in $(N - L - G - E)!$ ways.

So, the overall contribution of y is $y \cdot L \cdot \binom{x-2}{L-1} \cdot \binom{N-x}{G} \cdot (L - 1)! \cdot G! \cdot (N - L - G - E)!$. The multiplier is the same for all y , so after computing it once, it can be multiplied by the sum of all $y < x$ to get their overall contribution in $O(1)$. Similarly, the contribution of all $y > x$ can be computed in $O(1)$ time. Also, if $E = 1$ then $p_i = x$ must hold, so compute the contribution of x itself in similar fashion.

Notice that we now have a solution in $O(N)$ time for a single query: fix each index to be the terminating point of the binary search and compute the contribution in $O(1)$ as above.

To optimize this further, observe that the actual index i didn't matter at all: we only cared about L, G and E . In particular, if multiple different indices had the same (L, G, E) triplet, they'd all have the same contribution to the answer. Further, note that there can only be $O(\log(N)^2)$ distinct such triplets at all, since $0 \leq L + G + E \leq \lceil \log(N) \rceil$ and $0 \leq E \leq 1$.

So we can first simulate a single binary search on the entire array, to find all possible (L, G, E) triplets and their counts. Then, for each query, compute the contribution of each triplet multiplied by its frequency, and sum them up to obtain the final answer.

Time complexity: $O(N \cdot \log(N) + M \log(N)^2)$

D. Dexter's Laboratory

Author: Alexandru Gheorghies
Solved by: 21/21
First to solve: *KoRoVa!*

The problem can be summarized as follows:

- Given n and k , find the smallest number of edges in a connected graph with n nodes such that the shortest path between every pair of nodes is at most k .

It is trivial to see that if $k = 1$ then we need a complete graph. As such, the answer is $\frac{n \cdot (n-1)}{2}$.

Obviously, the graph needs to be connected (we need a path between any two cameras), so we cannot have less than $n - 1$ edges.

Otherwise, if $k \geq 2$, we can add an edge between the first node and every other node u ($2 \leq u \leq n$). The shortest path between every pair of nodes in this graph is at most $2 \leq k$.

This graph has $n - 1$ edges, which is the smallest number of edges possible (since the graph needs to be connected). As such, the answer for $k \geq 2$ is $n - 1$.

Time complexity: $O(1)$

E. Ed, Edd n Eddy

Author: Adrian Miclăuș
Solved by: 13/21
First to solve: Yahor Dubovik (mhq)

Fix an end j . Among all unique substrings ending at j with starts $i_1 < i_2 < \dots < i_k$, only the rightmost one i_k can be special. Otherwise $s[i_k..j]$ would be a proper substring of some other unique $s[i_t..j]$, contradicting the definition.

Let SA be the suffix array of s , $rank[p]$ its inverse, and LCP the array of lcp-values for adjacent suffixes in SA . Pad LCP by setting $LCP[0] = LCP[n] = 0$.

For the suffix starting at position p , define $L(p) \stackrel{\text{def}}{=} \max(LCP_{rank[p]}, LCP_{rank[p]+1})$.

$L(p)$ is the longest ℓ such that the prefix $s[p..p+\ell-1]$ occurs at least twice in s . Hence, if $p+L(p) < n$, then the length- $(L(p)+1)$ prefix $s[p..p+L(p)]$ is unique, and it ends at $j = p+L(p)$.

. For the final chosen start i at end j :

- **Left side repeats:** by definition of $L(i)$, $s[i..j-1]$ repeats.
- **Right side repeats:** we keep only the rightmost start for each end j ; thus $s[i+1..j]$ cannot be unique, so it repeats.

Any proper substring of $s[i..j]$ lies inside $s[i..j-1]$ or $s[i+1..j]$ and therefore repeats. Hence $s[i..j]$ is special.

Algorithm 1 Build SP

```

1: Build  $SA$ ,  $rank$ ,  $LCP$ ; set  $LCP[0] \leftarrow 0$  and append  $LCP[n] \leftarrow 0$ 
2: Initialize array  $SP[0..n-1] \leftarrow -1$ 
3: for  $r = 0$  to  $n-1$  do
4:    $p \leftarrow SA[r]$ 
5:    $L \leftarrow \max(LCP[r], LCP[r+1])$ 
6:    $j \leftarrow p + L$ 
7:   if  $j < n$  then
8:      $SP[j] \leftarrow \max(SP[j], p)$  ▷ keep rightmost start for this end
9:   end if
10: end for

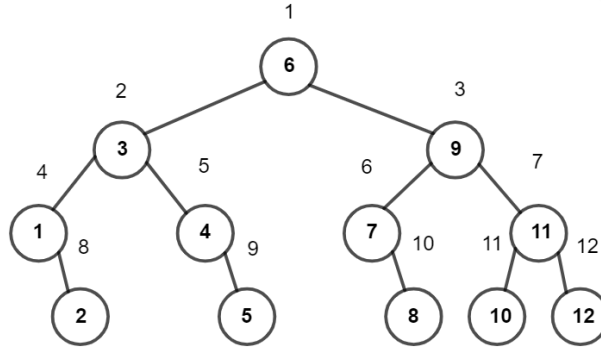
```

Building SA and LCP can be done in $O(n \log n)$ or $O(n \log^2 n)$. The final pass is $O(n)$. Memory is $O(n)$.

F. (The Marvelous Misadventures of) Flapjack

Author: Alexandru Gheorghies
Solved by: 21/21
First to solve: *Segfault go BRRRR*

To better understand the solution, let's consider $n=12$.



The final array is the BFS traversal starting from the root of the following binary tree:

This binary tree was generated by starting with the root $m = \lfloor \frac{n+1}{2} \rfloor = 6$. Each node $m = \lfloor \frac{l+r}{2} \rfloor$ has up to two children: $m_1 = \lfloor \frac{l+(m-1)}{2} \rfloor$ and $m_2 = \lfloor \frac{r+(m+1)}{2} \rfloor$.

If our given integer k is in a complete layer, then we can employ a binary search-type algorithm to find the position of k

```

1  l = 1, r = n, position = 1, visited_nodes = 1
2  while(visited_nodes <= n){
3      m = (l + r) / 2
4
5      if(k == m){
6          print position
7          return
8      }
9
10     if(k < m) position = position * 2
11     else position = position * 2 + 1
12
13     visited_nodes = visited_nodes * 2 + 1
14 }
15 
```

This algorithm has the added benefit of not printing anything if k is not in a complete layer. In this case, the position of k in the last layer can be determined from $position$, although this isn't very straightforward. If the binary tree were complete, then the position of k in the last layer would be $position - \frac{visited_nodes-1}{2}$, since there are $\frac{visited_nodes-1}{2}$ nodes on the other layers, all of which will be printed before k .

By looking at the missing nodes from the binary tree for $n = 12$, one may make the crucial observation that the general case position for k in the last layer is equal to $k - (position - \frac{visited_nodes-1}{2})$.

In conclusion, the final position for k will be $k - (position - \frac{visited_nodes-1}{2}) + \frac{visited_nodes-1}{2} = k + visited_nodes - position - 1$.

Time complexity per testcase: $O(\log(n))$

G. (The Amazing World of) Gumball

Author: Alexandru Gheorghies
Solved by: 6/21
First to solve: *Infinity*

The intended solution is just an optimization of an N^2 greedy solution. A good greedy algorithm should be implemented recursively, as such: Given a connected component of the tree, find a minimal chain (i.e. if the nodes used are sorted then that array would be lexicographically minimal).

When iterating in a component, get the node with the smallest label. Then, root the component in this minimum. To extend a chain, we get the minimum on each subtree, and we will try to extend in those that would eventually yield the smallest label. After selecting the nodes that would be a candidate chain, we erase them from the tree, thus remaining with some other subtrees, in which we will solve in a similar fashion. After extracting all the candidate chains, we then proceed to color them according to the smallest node included in it (i.e. the chain with the node 1 will be colored in the color 1, and thus all nodes on this chain will get color 1, then the chain with the next minimum (let's say 4) will be colored in 2, and so on and so forth).

Observe that a really big issue in our solution is that we constantly have to run through entire sub-components to get the minimal value in them, then we have to find the minimal value in subtrees etc. Optimizing this could be done using segment trees on any linearization of the tree (i.e. Preorder, Euler Tour, etc.). The only thing we have omitted so far is the fact that in the $O(N^2)$ solution, we rely on *rerooting* the component we are solving. After we create a chain, the node with the lowest depth (with respect to the first selected root) might not be the "root" of the component, thus the subtree remains with some "holes". How can we simulate the rerooting of the linearization?

Answer: we don't have to.

For further simplicity, let's call the nodes that haven't been yet colored "*active*", and consequently the nodes that have been given a chain called "*inactive*". Active nodes will be represented in the segment tree with their corresponding value, such that if we try to get the minimal value on a segment, they could be candidates, and inactive nodes will be represented in the segment tree by some absurd value (such as 10^9), so they could never be candidates again

As long as we are solving a subtree (or more rather, a $[L, R]$ segment) for which all active nodes that appear in the corresponding segment of the subtree in the linearization form a connected component, then there is nothing to worry about. To maintain this property, we suggest to write the program as follows:

Select the minimal node in $[L, R]$. Then, define a procedure in which you extend a chain that contains some node u . We go through each of its active neighbours. To get the minimal node in a subtree corresponding to the father of u , we query everything *outside* the corresponding segment of u (with respect to $[L, R]$). We now recur in the node that yields the minimal continuation.

After thus creating the minimal possible chain, we try to liquidate each subtree whose root is a neighbour of any node on this newly formed chain and its depth is greater than its neighbour's on the chain. We do this in a similar fashion as before.

And now, the most important step of all: if we denote with lca the node on the chain with minimal depth, then we can safely say that after the previous step, all nodes that are descendants of lca are now **inactive**. Thus, we can now try to create another candidate chain in the segment $[L, R]$, because so far, all we did is to erase the entire subtree of lca , so the graph formed by all active nodes in $[L, R]$ still form a connected component.

As for time analysis, each node could be only once a minimum value in a subtree, thus finding these values is $O(N \cdot \log(N))$, some nodes could be potential candidates to extension only $d(u)$ times, thus this step yields a time complexity of $O(N \cdot \log(N))$. Traversing some final candidate chains happens only once, and since every node should be in a chain, the time complexity from this step is $O(N)$. The

rest that happens is mere recursion, so it is already included in the calculations. Finally, atributing *final* colors to each chain is $O(N \cdot \log(N))$, since we have to sort each chain by its minimal value.

Finally, the total time complexity is $O(N \cdot \log(N))$ (with a pretty bad constant)

H. Hot Wheels: Battle Force 5

Author: Alexandru Gheorghies
Solved by: 17/21
First to solve: *Kindergarten Timelimit*

Let's try to solve a single query (u_i, v_i, k_i) first. Without loss of generality, we will assume that $u \leq v$, so we're only moving rightwards.

The natural greedy strategy is to move right using the existing portals as much as possible, and then teleport k to the right. It's fairly easy to see that this greedy strategy is also optimal.

This gives us a solution in $O(n)$ per query. Note that it can be improved slightly to $O(\frac{n}{k})$ by moving along existing roads quicker: for each i , precompute $\text{jump}[i]$ to be the farthest vertex to the right of i that can be reached using only existing roads - then starting from u you can alternately move to $\text{jump}[u]$ and $u + k$.

Every two moves increases the distance moved by $k + 1$, so we'll find the answer in $\leq O(\frac{n}{k})$ steps. If k is large enough (say, $k > \sqrt{n}$), doing this for each query would be fast enough. For small k however, it's still too slow, and we need a different strategy.

Let's use the fact that all queries are known in advance, and solve the problem offline. We will fix a value of k , and we'll try to answer all queries with this k reasonably quickly.

First, observe that for any query (u, v) , we could just as well answer the query $(\text{jump}[u], \text{jump}[v])$ since it'll have the same answer.

This simply skips the first and last steps of moving along public roads, which don't affect the answer anyway. Now, consider a graph on n vertices, with edges as follows:

For each i such that $\text{jump}[i] = i$, add the edge $i \rightarrow \text{jump}[i + k]$.

That is, we create a graph on the jump points alone denoting all "forced" movements. Then, for any $\text{jump}[u]$, if we want to get to $\text{jump}[v]$ the optimal solution (as noted right at the start) is to follow this graph till we reach a vertex that's $\geq \text{jump}[v]$.

The answer is the number of edges we follow. Here's the nice part: this graph is a tree, with vertex N being its root! Also, for any vertex u of the tree, the $n \rightarrow u$ path contains vertices in descending order.

So, for a query $(\text{jump}[u], \text{jump}[v])$ what we want is really just the lowest ancestor of $\text{jump}[u]$ that's $\geq \text{jump}[v]$.

This is quite easy to compute quickly: for example, you can perform a DFS on the tree and maintain the path to each vertex and then binary search on this path.

Building the tree is $O(n)$, and each query gets processed exactly once in $O(\log(n))$

Finally, simply combine both solutions! If $k > \sqrt{n}$ (or some similarly large constant), do the "brute force" jumping solution, which terminates quickly.

Then, for each $k = 1, 2, 3, \dots, \sqrt{n}$, answer all queries with such k by using the second method.

Time complexity: $O((n + q) \cdot \sqrt{n})$

I. Infinity Train

Author: Gabriel Turbincă
Solved by: 11/21
First to solve: *xXx_kasza_jaglana_xXx*

We can compute the answers for $n \leq 6$ using bruteforce, since we will find solutions with pretty small numbers.

Let's now say that we have a solution for n , which is $x_0, x_1, \dots, x_n$ and y_0, y_1, y_2, y_3, y_4 is a solution for 4. Then, $y_0 \cdot x_0, y_1 \cdot x_1, y_1 \cdot x_2, \dots, y_1 \cdot x_n, y_2 \cdot x_0, y_3 \cdot x_0, y_4 \cdot x_0$ is a valid solution for $n + 3$. By having the solutions for 4, 5 and 6 precomputed, we can construct the solution inductively for all the n and this will be enough to pass.

Time complexity: $O(N^3)$

Memory complexity: $O(N^2)$

J. Johnny Test

Author: Adrian Miclăuș
Solved by: 20/21
First to solve: *Seems to be $O(k!)$*

Observation: If some length-3 factor $A_i A_{i+1} A_{i+2}$ occurs at least three times in the sequence, the answer is NO.

Proof. In each occurrence of the same 3-gram XYZ , the middle index must be covered by either the left pair XY or the right pair YZ . Since all picked 2-grams must be globally distinct, the first two occurrences can use XY and YZ once each. A third occurrence would force reusing one of them, violating distinctness. □

Reduction to 2-SAT

Create a Boolean variable p_i for each position $i \in \{1, \dots, n-1\}$, meaning:

$$p_i = \text{“we select the pair } (A_i, A_{i+1})\text{”}.$$

Add two families of clauses:

Coverage. Every position must be covered by at least one adjacent pair; endpoints are forced:

$$\begin{aligned} C_1 &: (p_1), \\ C_n &: (p_{n-1}), \\ C_i &: (p_{i-1} \vee p_i) \quad \forall i, 2 \leq i \leq n-1. \end{aligned}$$

Distinctness. Equal 2-grams cannot both be taken. For any $1 \leq i < j \leq n-1$ with $(A_i, A_{i+1}) = (A_j, A_{j+1})$, add

$$B_{ij} : (\neg p_i \vee \neg p_j).$$

Let F be the conjunction of all C and B clauses. Then F is satisfiable iff the required selection exists.

Size bound. After the 3-gram test, every 3-gram occurs at most twice. Fix a 2-gram s ; any other occurrence of s is either followed by some $c \in \Sigma$ (forming a 3-gram sc) or is a suffix. Each sc can appear at most twice, so each occurrence of s conflicts with at most $2|\Sigma| + 1$ others. Hence the number of B_{ij} clauses is $O(n|\Sigma|)$ and the whole construction (and 2-SAT solve) runs in $O(n|\Sigma|)$ time.

K. Krypto the Superdog

Author: Alexandru Gheorghies
Solved by: 19/21
First to solve: *xXx_kasza_jaglana_xXx*

Observation: There exists an optimal solution where all the chosen intervals are nested.

Proof : If we have at most 1 intervals then this is obviously true. Otherwise, choose two intervals $[l_1, r_1]$ and $[l_2, r_2]$. We assume without loss of generality that $l_1 < l_2$, and are left with four cases.

I. $r_2 \leq r_1$. In this case the intervals are already nested.

II. $l_2 \leq r_1$. In this case we can replace these two intervals with $[l_1, r_2]$ and $[l_2, r_1]$.

III. $l_2 = r_1 + 1$. In this case we replace thse two intervals with a single interval $[l_1, r_2]$.

IV. $l_2 > r_1 + 1$. In this case we replace these two intervals with $[l_1, r_2]$ and $[r_1 + 1, l_2 - 1]$.

In each case, the new intervals allow the same modifications as the old ones, are nested, and do not increase the total number of intervals.

Since we know that an optimal solution with nested intervals exists, we can look for it using two pointers by maintaining the current smallest interval that contains all the non-optimal values. Let this interval be $[L, R]$, it becomes either $[L + 1, R]$ or $[L, R - 1]$ depending on which end becomes optimal first.

L. Lego Ninjago: Masters of Spinjitzu

Author: Alexandru Gheorghies
Solved by: 19/21
First to solve: Xeppelin

We can manually compute the answer for $n \leq 5$. From now on, we will assume $n \geq 6$.

The first observation is that if we want a certain master to win, we will try to minimize the strength remaining to the left and the right of him, as we will then need to subtract those numbers from the strength of our master.

The left side can be treated in a straight-forward manner, because we can always reach a state where only a strength of 0 or 1 remains, depending on the parity of the sum of numbers on that prefix.

Computing the minimum strength left on the right side is not that trivial, but we can observe that we don't really need to minimize that number, but rather make it at most $x - 2$, if we want position x to win. (we will treat positions 1 and 2 separately).

We are left with the following problem: Given a list of consecutive numbers that start at a number a , determine if we can reach a state where only a number less than or equal to $a - 3$ remains (considering $a = x + 1$). It is obvious that if the length of the list is even, we can always have 0 or 1 remaining, so we are only interested in odd lengths.

Proposition 1: It is possible to reach a number less than or equal to $a - 3$ if and only if the length of the list is even, or if it is greater than or equal to 5.

Proof: It is possible to reach $a - 4$ starting from the list $[a, a + 1, a + 2, a + 3, a + 4]$:

$$[a, a + 1, a + 2, a + 3, a + 4] \rightarrow [a, a + 1, 1, a + 4] \rightarrow [a, a, a + 4] \rightarrow [a, 4] \rightarrow [a - 4]$$

For odd length lists, adding an even amount of numbers to the right of $[a, a + 1, a + 2, a + 3, a + 4]$ can only decrease our answer. If the length is 1 it's obvious we can't make it any smaller, and for a length of 3 the best we can achieve is $a - 1$. Thus, we have proven the proposition for any $a \geq 4$ and since we said that we will deal with $x = 1$ and $x = 2$ separately, our proof is complete.

The above statement tells us that any x except 1, 2, $n - 3$ and $n - 1$ is always a winning position that needs to be counted towards our answer. That is because we can always reach a state of the form y, x, z with $y \leq 1$ and $z \leq x - 2$. For $n - 3$ and $n - 1$ we cannot make the number to their right sufficiently small, so they will never be counted towards our answer.

Proposition 2: If $n \equiv 1 \pmod{4}$ or $n \equiv 2 \pmod{4}$ positions 1 and 2 are winning, otherwise they are not.

Proof: If $n \equiv 3 \pmod{4}$ or $n \equiv 0 \pmod{4}$, the sum of numbers to the right of 1 and 2 is odd, so the minimum value achievable on the right side is 1, thus making it impossible for both 1 and 2 to win. Let's analyze the other 2 cases for position 1:

- If $n \equiv 1 \pmod{4}$ we are left with a list of length divisible by 4 to the right of 1, which can always be reduced to 0.
- If $n \equiv 2 \pmod{4}$ we are left with the number 2, 4 consecutive numbers, and a list of length divisible by 4. All the numbers to the right of the number 2 will be paired to form ones, the first 2 ones will be subtracted from the number 2 and the other ones will cancel each other, leaving the number 0.

The cases for position 2 will be treated as follows:

- If $n \equiv 2 \pmod{4}$ we are left with a list of length divisible by 4 to the right of 2, which can always be reduced to 0.
- If $n \equiv 1 \pmod{4}$, since we assumed that $n \geq 6$, we now have $n \geq 9$. We will group all the numbers to the right of 3 into pairs that make the number 1, the first three pairs will be subtracted from 3 and the other pairs (there will be an even number of these) will cancel eachother, thus leaving the number 0 overall.

We have now proven everything, so the answer for any $n \geq 6$ is $n - 2$ if $n \equiv 1 \pmod{4}$ or $n \equiv 2 \pmod{4}$ and $n - 4$ otherwise.

Time complexity per testcase: $O(1)$

Memory complexity: $O(1)$

M. Mao Mao: Heroes of Pure Heart

Author: Alexandru Gheorghies
Solved by: 21/21
First to solve: *xXx_kasza_jaglana_xXx*

We take $a_i = i$ for $1 \leq i \leq n - 3$.

If $1 \oplus 2 \oplus 3 \oplus \dots \oplus (n - 2) \neq 0$ we take $a_{n-2} = n - 2$, otherwise we take $a_{n-2} = n - 1$.

We can then iterate with a variable x from 1 to $+\infty$, until both x and $x \oplus a_1 \oplus a_2 \oplus a_3 \oplus \dots \oplus a_{n-2}$ are available.

Then, we can set $a_{n-1} = x$ and $a_n = x \oplus a_1 \oplus a_2 \oplus a_3 \oplus \dots \oplus a_{n-2}$.

Time complexity: $O(n)$, since $x \leq 4 \cdot n$.